

DOCUMENT DE SPÉCIFICATIONS

Stage de 2^{ème} Année Informatique – ENSEIRB-MATMECA

Sujet : Analyse temps réel multi-caméras de défauts industriels à partir de descriptions textuelles et visuelles. Dimensionnement matériel/logiciel de l'IA.

Étudiant : Pedro RUSSO (2^{ème} Année Informatique, ENSEIRB-MATMECA)
Organisme d'accueil : Inria Centre Université de Bordeaux / LaBRI (UMR 5800)
Équipe de recherche : Équipe-projet TOPAL
Encadrant d'accueil : Denis BARTHOU (Professeur à Bordeaux INP, Chercheur Inria/LaBRI)
Dates du stage : 10 juin 2026 – 9 septembre 2026

1. Présentation de l'Organisme d'Accueil et de l'Équipe

L'Inria (Institut National de Recherche en Sciences et Technologies du Numérique) est l'organisme national français dédié à la recherche fondamentale et appliquée en informatique et en mathématiques appliquées. Le stage s'effectue au sein du centre **Inria Centre Université de Bordeaux**.

Plus précisément, le stage est hébergé au sein de l'équipe-projet **TOPAL**. Cette équipe de recherche est reconnue pour ses travaux de pointe en calcul haute performance (HPC), optimisation de code, systèmes d'exécution sous contraintes matérielles, compilation et accélération sur architectures hétérogènes (CPU/GPU).

Le stage est encadré par le Professeur **Denis BARTHOU**, chercheur au sein de cette équipe. Cet environnement scientifique offre un cadre idéal pour associer les techniques émergentes d'Intelligence Artificielle en vision par ordinateur aux problématiques d'optimisation matérielle et logicielle bas niveau.

2. Contexte, Problématique et Description du Sujet

Le contrôle qualité automatique sur les chaînes de production industrielles exige d'intercepter instantanément tout défaut de fabrication (rayures, fissures, déformations) à partir de flux vidéo multi-caméras. Dans ce contexte, l'utilisation de modèles d'IA embarqués constitue un levier majeur : elle permet de traiter les données au plus près des capteurs sans dépendre d'une connexion réseau vers un serveur distant, garantissant ainsi une latence minimale et une confidentialité totale des données.

Le sujet de stage s'intéresse à la détection temps réel d'anomalies multi-caméras en combinant des descriptions visuelles et textuelles (approche Zero-Shot via des modèles fondateurs comme WinClip), ainsi qu'à des architectures dédiées au transfert d'apprentissage non supervisé (comme EfficientAD). L'enjeu central réside dans le co-dimensionnement matériel et logiciel : il s'agit d'adapter et d'optimiser ces modèles profonds particulièrement gourmands en calcul afin qu'ils s'exécutent de façon fluide et déterministe sur des plates-formes embarquées contraintes, spécifiquement les cartes **NVIDIA Jetson Orin**.

3. Illustration de la Problématique et du Pipeline Système

Afin de distinguer clairement la phase de développement et de dimensionnement de la phase d'exécution en usine, le pipeline système se décompose en deux phases complémentaires illustrées par les Figures 1 et 2.

3.1 Phase Hors-Ligne : Entraînement, Export et Compilation du Moteur TensorRT

La Figure 1 présente la chaîne de préparation hors-ligne. Dans cette phase, les images ne proviennent pas de caméras physiques mais d'une base de données d'images pré-enregistrées (dataset MVTec AD). Les modèles de détection d'anomalies (tels qu'EfficientAD ou WinClip) y sont entraînés ou configurés sous PyTorch, puis exportés au format intermédiaire ONNX. Enfin, l'outil trtexec compile ce graphe sous la forme d'un fichier moteur binaire autonome (fichiers **.engine**) optimisé en précision FP16/INT8 pour l'architecture GPU cible.

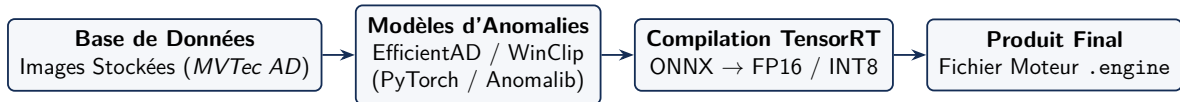


FIGURE 1 – Phase Hors-Ligne : Pipeline d'entraînement, d'exportation et de compilation du moteur TensorRT.

3.2 Phase En-Ligne : Déploiement Embarqué et Inférence Temps Réel

La Figure 2 illustre le déploiement réel sur le site de production. Les flux d'images sont capturés en direct par des caméras d'inspection physiques. Le runtime d'inférence, écrit en Python via l'API PyCUDA, charge le moteur compilé **.engine** directement en mémoire GPU embarquée (NVIDIA Jetson Orin) sans passer par PyTorch. Le système génère instantanément une décision binaire **OK** / **NOK** (pièce conforme ou défectueuse) basée sur un seuil (politique Zéro Défaut), garantissant un débit maximal.

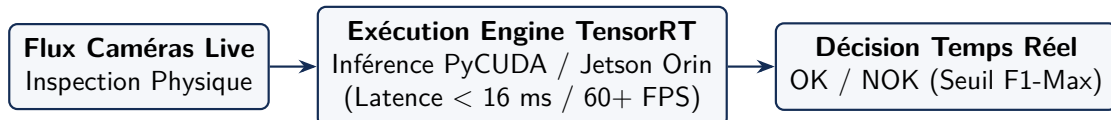


FIGURE 2 – Phase En-Ligne : Déploiement en production sur NVIDIA Jetson Orin pour décision temps réel OK/NOK.

4. Démarche Scientifique et Objectifs Techniques

Afin de répondre à la problématique du dimensionnement matériel/logiciel, le travail s'articule autour de plusieurs objectifs scientifiques et techniques majeurs :

1. Évaluation et benchmarking de modèles d'anomalies en Zero-Shot :

Mise en œuvre et étude comparative de modèles de pointe issus de la bibliothèque *Anomalib* (notamment WinClip et EfficientAD) sur des jeux de données industriels de référence tel que *MVTec AD*. Il s'agit d'analyser le compromis entre la capacité de généralisation sans réentraînement (Zero-Shot) et la précision de segmentation des défauts.

2. Optimisation bas niveau et accélération matérielle sur NVIDIA Jetson Orin :

Exportation des graphes de calcul PyTorch vers le format d'échange ONNX, suivie d'une étape de compilation sous forme de moteurs exécutables TensorRT ('engine'). Cette phase intègre la

quantification en précision demi-flottante (FP16) et entière (INT8) afin de maximiser l'utilisation des unités de calcul vectoriel et de contourner la saturation du processeur central ARM.

3. Développement d'un runtime d'inférence à haute performance :

Conception de scripts d'inférence en Python s'appuyant sur l'API bas niveau **PyCUDA**. L'accent est mis sur la gestion avancée de la mémoire afin d'éliminer les goulots d'étranglement lors du transfert mémoire entre le CPU et le GPU embarqué.

4. Calibration et politiques de décision zéro défaut :

Mise au point d'une méthodologie rigoureuse de calibration des seuils d'anomalie (seuil F1-Max) sur des catégories de référence. L'objectif est de garantir un taux de rappel de 100% (aucune pièce défectueuse manquée / aucun faux négatif), répondant ainsi aux exigences strictes des cahiers des charges industriels.

5. Adéquation avec la Formation de 2^{ème} Année Informatique

Ce sujet de stage s'inscrit en adéquation avec le programme de la deuxième année du cycle ingénieur Informatique de l'ENSEIRB-MATMECA. Il permet de mobiliser et d'approfondir un ensemble de compétences enseignées à l'école :

- **Génie logiciel et développement système** : Conception de pipelines logiciels modulaires et robustes en Python, automatisation d'expérimentations via des scripts Bash et gestion de versions.
- **Intelligence Artificielle** : Manipulation de frameworks de Deep Learning (PyTorch, Anomalib).
- **Architecture des ordinateurs** : Optimisation sous contraintes matérielles, programmation GPU (CUDA / TensorRT).
- **Méthodologie expérimentale** : Mesure précise des performances (latence en millisecondes, débit en images par seconde, occupation mémoire), analyse statistique des métriques de classification (AU-ROC, AU-PRO, rappel, précision).

6. Conditions de Travail et Intégration

Le stage se déroule dans de bonnes conditions logistiques et scientifiques au sein des locaux d'Inria à Talence. Je dispose d'un espace de travail équipé d'un poste informatique performant connecté au réseau du laboratoire, ainsi que d'un accès direct à plusieurs bancs de test embarqués équipés de cartes **NVIDIA Jetson Orin**.

L'encadrement par le Professeur Denis BARTHOU s'effectue au travers d'échanges hebdomadaires réguliers. Ces discussions permettent de faire un point sur l'avancement des résultats, d'orienter les choix d'optimisation et d'aborder les verrous méthodologiques. L'intégration au sein de l'équipe TOPAL s'est faite très naturellement.

7. Bilan Intermédiaire et Planning Prévisionnel

L'avancement du stage respecte le calendrier initialement prévu et se décompose en trois grandes phases :

Phase 1 – Prise en main et benchmarks baselines (Réalisée) :

Mise en place de l'environnement de développement Anomalib, structuration du dataset MVTec AD et évaluation initiale des modèles PyTorch sur CPU/GPU standard. Cette première phase a permis d'établir la ligne de référence.

Phase 2 – Optimisation et compilation TensorRT (Réalisée / En cours) :

Mise en place de la chaîne d'export ONNX et de compilation des moteurs TensorRT en FP16. L'élimination des surcoûts d'autocast dynamique et la mise en œuvre de transferts mémoire optimisés via PyCUDA ont permis d'atteindre une latence d'inférence de 14 à 16 ms par image, portant le débit à plus de 60 FPS.

Phase 3 – Multi-caméras, calibration et valorisation (À venir) :

Extension de la chaîne d'inférence au traitement simultané de plusieurs flux vidéo multi-caméras, finalisation de la calibration automatique des seuils d'anomalie sur l'ensemble des catégories industrielles, et rédaction du rapport final de stage.